



FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE DE
COIMBRA

Mestrado em Engenharia Informática
Faculdade de Ciências e Tecnologia da Universidade de Coimbra

Arquitetura de Software

Guardian | ASR

Eduardo Marques - 2022231584
João Cardoso - 202222301
Tiago Marques - 2022210638

Conteúdo

1	Introdução	3
1.1	Use Cases	3
1.2	Diagrama de Use Cases	13
1.3	Objetivos de Qualidade	14
1.4	Stakeholders	14
2	Restrições	14
2.1	Restrições Técnicas	14
2.2	Restrições de Negócio	15
3	Contexto e Âmbito	16
3.1	Contexto de Negócio	16
3.2	Contexto Técnico	16
4	Estratégia de Solução	18
4.1	Decisões Tecnológicas	18
4.2	Decisões sobre o Design da Arquitetura	18
4.3	Decisões sobre Objetivos de Qualidade	19
4.4	Processo de Desenvolvimento da Arquitetura	19
5	Visão do Contexto (C1)	21
6	Visão de Containers (C2)	22
7	Visão de Componentes (C3)	23
8	Visão de Deployment	24
8.1	Estratégia de Deployment	24
8.2	Requisitos de Infraestrutura	24
8.3	Diagrama de Deployment	25
9	Conceitos Transversais	26
9.1	Monitorização e Tolerância a Falhas	26
9.2	Segurança e Privacidade	26
10	Decisões Arquiteturais	27
10.1	DA-01: Adoção do Estilo Arquitetural Microkernel + Event-Driven	27
10.2	DA-02: Escolha de Python como Linguagem Principal	28
10.3	DA-03: SQLite como Base de Dados Local	29
10.4	DA-04: MQTT como Protocolo de Comunicação Interna	29
10.5	DA-05: PWA para Interface do Idoso	30
10.6	Tabela com o Resumo das Decisões Arquiteturais	30
11	Atributos de Qualidade	32
AQ-01	Reliability	32
AQ-02	Availability (Disponibilidade)	32
AQ-03	Performance (Desempenho)	32
AQ-04	Integrability (Integração)	33
AQ-05	Usability (Usabilidade)	33

AQ-06 Security (Segurança de Dados)	34
AQ-07 Testability (Testabilidade)	34
12 Riscos e Dívida Técnica	36
12.1 Riscos Identificados	36
12.2 Dívida Técnica	37
12.2.1 Dívida Técnica Intencional	37
12.3 Oportunidades de Melhoria	38
12.3.1 Melhorias de Arquitetura	38
12.3.2 Experiência do Utilizador	38
12.3.3 Conformidade e Segurança	39
13 Glossário	40
14 Conclusão	42

1 Introdução

O envelhecimento da população constitui um desafio societal crescente, associado ao aumento significativo do risco de quedas e acidentes domésticos, particularmente entre idosos que residem sozinhos. Para mitigar estes riscos e proporcionar maior tranquilidade a familiares e cuidadores, propõe-se o **Guardian**: um sistema de assistência ativa baseado em IoT que privilegia a privacidade do idoso enquanto promove a sua qualidade de vida através de intervenções proativas e não intrusivas.

O sistema funciona através de um modelo de duas fases: uma fase inicial de aprendizagem passiva das rotinas quotidianas do idoso, seguida de uma fase de intervenção automatizada e contextualizada. Sensores heterogêneos distribuídos pela habitação recolhem dados que permitem ao sistema aprender padrões comportamentais individuais.

1.1 Use Cases

UC-01: Ativar Corredor de Iluminação

Descrição

O sistema ativa uma iluminação suave e indireta em um percurso predefinido. O objetivo é mitigar o risco de queda.

Atores Principais

Idoso (iniciador passivo), Sistema.

Gatilho

Sensor de movimento posicionado junto à cama regista ausência durante período 'Noite' configurado.

Fluxo de Eventos Principal

Ator: Idoso	Ator: Sistema
1. Abandona a cama.	2. Deteta a ausência do Idoso, e ativam as luzes do percurso com intensidade reduzida (ex: 20%).
3. Passa pelo corredor.	
4. Retorna à cama.	5. Deteta a presença, valida o regresso e dá ordem para extinguir a iluminação.

Fluxos Excepcionais e Alternativos

- **2a.** A funcionalidade encontra-se "Inativa" (definido no UC-06). O Sistema regista o evento (para o UC-05), mas não acende as luzes.
- **2b.** A hora atual não está no período "Noite". O Sistema ignora a iluminação e regista o motivo.
- **5a.** O Idoso não volta à cama (ex.: dirige-se a outra divisão). O Sistema desliga a iluminação do percurso inicial após 15 minutos (configurável) e regista o evento.

- **5b.** Falha na ativação das luzes no percurso. O Sistema registra o ocorrido e notifica o Cuidador.

Pré-condições

- O Sistema Guardian está operacional.
- Os sensores e as luzes do percurso estão operacionais.
- A automação "Caminho Seguro" está "Ativa" (UC-06).
- O relógio do sistema indica o período "Noite".

Pós-condições

- A iluminação do percurso está desativada.
- Evento registrado.

Dependências

UC-06 (Gestão de preferências de automação).

UC-02: Ativar Despertador

Descrição

O sistema simula o nascer do sol, aumentando gradualmente a intensidade e alterando a temperatura da cor das luzes do quarto.

Atores Principais

Sistema (iniciador), Idoso (recetor).

Gatilho

Horário programado no "Despertador" (ex.: 07:30) corresponde à hora atual.

Fluxo de Eventos Principal

Ator: Sistema	Ator: Idoso
1. Atinge o horário programado para despertar (ex: 07:30) e valida que a automação está "Ativa" (UC-06).	
2. Inicia a mudança gradual das luzes do quarto (intensidade e cor) durante um período predefinido.	3. O Idoso acorda.

Fluxos Excepcionais e Alternativos

- **2a.** A funcionalidade está "Inativa" (UC-06). O Sistema não executa a automação.
- **2b.** O Idoso levanta-se (detetado pelo sensor da cama) antes ou durante a execução. O Sistema interrompe o "Despertador".

Pré-condições

- O Sistema está operacional.
- Um horário de despertar está configurado.
- As luzes do quarto (com controlo de intensidade e cor) estão operacionais.
- A automação "Despertador" está "Ativa"(UC-06).

Pós-condições

- As luzes do quarto atingem a intensidade predefinida para o despertar.
- As luzes do quarto atingem a temperatura de cor predefinida para o despertar.

Dependências

UC-06 (Gestão de preferências de automação).

UC-03: Emitir Lembrete Visual

Descrição

O sistema emite um sinal visual discreto num horário programado assistindo o Idoso a lembrar-se de uma tarefa.

Atores Principais

Sistema (iniciador), Idoso (recetor).

Gatilho

Chegada do lembrete num horário configurado (UC-09).

Fluxo de Eventos Principal

Ator: Sistema	Ator: Idoso
1. Atinge o horário programado (configurado no UC-09) e valida que a automação está "Ativa"(UC-06).	
2. Ativação do lembrete visual.	3. Observa o lembrete e executa a ação.
	4. Aparece botão de "Confirmação" na interface do Idoso.
5. Recebe a confirmação e envia ordem para parar o Lembrete.	

Fluxos Excepcionais e Alternativos

- **4a.** O Idoso não confirma a ação após 10 minutos. O Sistema regista o evento como "não confirmado" e notifica o cuidador (visível no UC-07).

Pré-condições

- O Sistema Guardian está operacional.
- Existem lembretes configurados (UC-09).
- A automação "Lembretes" está "Ativa" (UC-06).

Pós-condições

- O lembrete visual é desativado.
- O sistema regista se a tarefa foi confirmada ou não.

Dependências

UC-09 (Configurar Lembretes) e UC-06 (Gestão de preferências de automação).

UC-04: Gerir Clima

Descrição

O sistema monitoriza e controla a temperatura ambiente, ou alerta o Cuidador em caso de risco.

Atores Principais

Sistema (iniciador).

Gatilho

Temperatura fora dos limites de conforto (mín./máx. configurados) num período superior a 30 minutos.

Fluxo de Eventos Principal

Ator: Sistema

1. Monitoriza continuamente a temperatura e deteta que saiu dos limites de conforto por um período prolongado.
 2. Valida que a automação está "Ativa" (UC-06).
 3. Tenta corrigir a temperatura enviando ordens para ativar o dispositivo de climatização.
 4. Continua a monitorizar. Se a temperatura for corrigida, o ciclo termina.
-

Fluxos Excepcionais e Alternativos

- **3a.** A funcionalidade está "Inativa"(UC-06). O Sistema não atua.
- **3b.** Dispositivo de climatização indisponível. O Sistema regista o incidente e notifica o Cuidador.
- **4a.** Se a temperatura não for corrigida e atingir um limiar de risco, é enviada uma notificação de alerta.

Pré-condições

- O Sistema está operacional.
- Limiares de conforto e risco estão definidos.
- A automação "Gestão de Clima"está "Ativa"(UC-06).

Pós-condições

- A temperatura da residência é mantida na zona de conforto, ou o Cuidador é alertado para um risco.

Dependências

UC-06 (Gestão de preferências de automação).

UC-05: Notificar Cuidador sobre Anomalia Comportamental

Descrição

O Sistema identifica um comportamento fora do normal do Idoso e notifica o Cuidador.

Atores Principais

Sistema (iniciador), Cuidador (recetor).

Gatilho

Deteção de um comportamento fora do normal.

Fluxo de Eventos Principal

Ator: Sistema	Ator: Cuidador
1. Recolhe dados e compara a atividade em tempo real com o perfil de referência do Idoso.	
2. Deteta um evento que excede o limite crítico.	
3. Classifica o evento e envia a notificação.	4. Recebe a notificação de alerta.
5. Regista o alerta no "Painel de Tranquilidade"(UC-07).	

Fluxos Excepcionais e Alternativos

- **2a.** O Sistema deteta um desvio ligeiro. O Sistema regista o comportamento, mas não gera um alerta.

Pré-condições

- O Sistema está operacional.
- O Sistema teve um período de aprendizagem inicial de pelo menos 1 semana.
- Os canais de notificação do Cuidador estão configurados.
- O Sistema confirma que o Idoso está em casa.

Pós-condições

- O Cuidador é notificado da potencial situação de emergência.
- O estado no "Painel de Tranquilidade" é atualizado para "Alerta Ativo".

Dependências

UC-07 (Consulta e visualização no painel).

UC-06: Gerir Automações (Interface do Idoso)

Descrição

O Idoso ativa ou desativa as funcionalidades numa interface simplificada.

Atores Principais

Idoso, Sistema.

Gatilho

O Idoso acede à sua interface.

Fluxo de Eventos Principal

Ator: Idoso	Ator: Sistema
1. Acede à "Interface do Idoso".	2. Apresenta a lista de funcionalidades e os seus estados atuais ("Ativo" ou "Inativo").
3. Seleciona uma funcionalidade e clica para alterar o estado.	4. Recebe a alteração.
	5. Guarda esta nova preferência do utilizador.
	6. Confirma a alteração na Interface do Idoso.

Fluxos Excepcionais e Alternativos

- **5a.** O Sistema não consegue guardar a alteração. O Sistema informa o Idoso que a alteração não pôde ser guardada.

Pré-condições

- O Idoso tem acesso à sua interface.
- A interface está ligada ao Sistema.

Pós-condições

- O estado da funcionalidade é guardado.

Dependências

UC-07: Consultar Painel de Tranquilidade (Interface do Cuidador)

Descrição

O Cuidador acede à aplicação para verificar o estado geral do Idoso.

Atores Principais

Cuidador, Sistema.

Gatilho

Autenticação bem-sucedida por parte do Cuidador (UC-08).

Fluxo de Eventos Principal

Ator: Cuidador	Ator: Sistema
1. Autentica-se (UC-08) e abre a aplicação.	2. Apresenta o "Painel de Tranquilidade" com as seguintes informações: • Estado Atual. • Referência da Última Atividade. • Histórico de Alertas Graves. • Configuração de lembretes.
3. Consulta o painel.	

Fluxos Excepcionais e Alternativos

- **2a.** O estado atual é de alerta. O painel exibe detalhes do alerta com várias ações possíveis.

Pré-condições

- O Cuidador está autenticado (UC-08).

Pós-condições

- O Cuidador fica informado sobre o estado do Idoso.

Dependências

UC-08 (Autenticação).

UC-08: Autenticar Cuidador

Descrição

O Cuidador acede à sua conta para consultar o "Painel de Tranquilidade"(UC-07) e gerir configurações (UC-09).

Atores Principais

Cuidador, Sistema Guardian.

Gatilho

O Cuidador tenta autenticar-se.

Fluxo de Eventos Principal

Ator: Cuidador	Ator: Sistema
1. Abre a interface da aplicação.	2. Apresenta um formulário de autenticação.
3. Insere as suas credenciais.	4. Valida as credenciais.
	5. Estabelece uma sessão segura.
	6. Apresenta o "Painel de Tranquilidade"(UC-07).

Fluxos Excecionais e Alternativos

- **4a.** Credenciais inválidas. O Sistema informa o Cuidador do erro e não inicia sessão.

Pré-condições

- O Cuidador possui uma conta válida.
- A conta do Cuidador está formalmente associada à conta do Idoso.

Pós-condições

- O Cuidador tem uma sessão ativa.

Dependências

UC-07 (apresentação do painel), UC-09 (gestão de lembretes).

UC-09: Configurar Lembretes (Interface do Cuidador)

Descrição

O Cuidador agenda lembretes visuais (que serão executados no UC-03) e envia-os para o Idoso.

Atores Principais

Cuidador, Sistema.

Gatilho

O Cuidador inicia a criação de um lembrete.

Fluxo de Eventos Principal

Ator: Cuidador	Ator: Sistema
1. Autentica-se (UC-08) e navega para a secção "Lembretes".	2. Exibe os lembretes atuais.
3. Seleciona "Criar Novo Lembrete".	4. Apresenta um formulário (Nome, Horário, Ação do lembrete, Recorrência).
5. Preenche o formulário e submete.	6. Valida os dados e guarda a configuração do novo lembrete.
	7. Atualiza a lista de lembretes na interface do Cuidador.

Fluxos Excepcionais e Alternativos

- **6a.** Dados inválidos. O Sistema informa o Cuidador do erro.
- O Cuidador pode também "Editar"ou "Eliminar"lembretes existentes (fluxo não detalhado).

Pré-condições

- O Cuidador tem uma sessão ativa (UC-08).

Pós-condições

- O novo lembrete está guardado e será executado pelo Sistema (UC-03) no próximo horário agendado.

Dependências

UC-08 (Autenticação) e UC-03 (Execução do lembrete).

UC-10: Associar Idoso ao Cuidador

Descrição

O sistema permite que um Cuidador se associe a um Idoso, estabelecendo uma relação formal entre ambos.

Atores Principais

Cuidador (iniciador), Sistema, Idoso (recetor).

Gatilho

O Cuidador inicia o processo de associação a um Idoso.

Fluxo de Eventos Principal

Ator: Cuidador	Ator: Sistema
1. Acede à secção de "Associar Idoso".	2. Apresenta um formulário para introduzir o identificador do Idoso.
3. Insere o identificador único do Idoso (ex: email ou código).	4. Valida o identificador e confirma a existência da conta do Idoso.
5. Aguarda confirmação.	6. Envia uma notificação ao Idoso solicitando a aprovação da associação.
	7. Armazena a relação (em estado pendente).

Fluxos Excepcionais e Alternativos

- **4a.** O identificador do Idoso não existe no sistema. O Sistema informa o Cuidador que nenhuma conta foi encontrada.
- **4b.** O Idoso já tem uma associação ativa com outro Cuidador. O Sistema informa o Cuidador que a operação não é permitida.
- **6a.** O Idoso rejeita a associação. O Sistema notifica o Cuidador e cancela a relação.

Pré-condições

- O Cuidador possui uma conta válida no Sistema Guardian.
- O Idoso possui uma conta válida e ativa no Sistema Guardian.
- O Idoso não tem atualmente um Cuidador associado (ou o Cuidador anterior foi removido).

Pós-condições

- A relação entre Idoso e Cuidador está estabelecida e ativa.
- O Cuidador tem acesso ao painel de monitorização do Idoso.
- O Cuidador pode gerir configurações e receber notificações relacionadas com o Idoso.

1.2 Diagrama de Use Cases

Como se observa na Figura 1, o ator *Idoso* interage principalmente com automações reativas (UC-01, UC-02, UC-03), enquanto o *Cuidador* atua na monitorização e configuração (UC-07, UC-08, UC-09). O diagrama destaca: (i) o limite do sistema (*Guardian*); (ii) os atores primários e o sistema externo *Home Assistant*; e (iii) dependências explícitas (p.ex., UC-07 e UC-09 incluem autenticação UC-08, e UC-01/UC-02/UC-04 respeitam preferências definidas no UC-06).

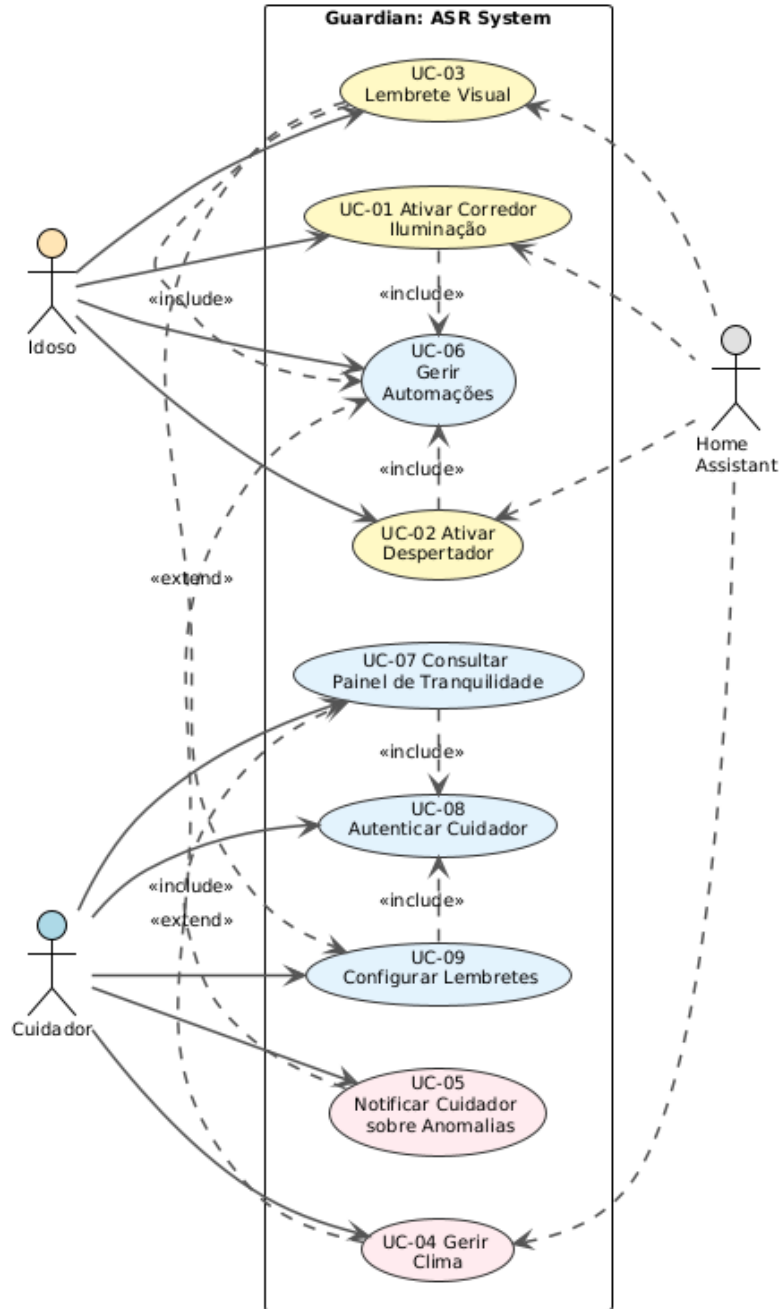


Figura 1: Diagrama de Use Cases do sistema Guardian

1.3 Objetivos de Qualidade

Os objetivos de qualidade prioritários da arquitetura são apresentados na tabela seguinte:

Objetivo de Qualidade	Prioridade	Pequena Descrição
Reliability	Alta	Garantir comportamento previsível e recuperação automática de falhas, minimizando interrupções no serviço.
Availability	Alta	Assegurar disponibilidade contínua (objetivo: 99,9% uptime anual).
Performance	Alta	Garantir que o sistema é rápido a agir e a detetar
Integrability	Média	O sistema deve ser modular com a API externa do Home Assistant
Security	Média	Todos os dados do sistema devem estar protegidos entre comunicações
Usability	Baixa	O sistema deve ser de fácil utilização, tanto para o idoso como para o cuidador
Testability	Baixa	Garantir que os desenvolvedores consigam testar o sistema sem ter esperas significativas

Nota: Os Quality Attributes Scenarios completos encontram-se detalhados na secção de Atributos de Qualidade deste relatório.

1.4 Stakeholders

Papel	Expectativas
Idoso	Receber apoio da aplicação, ativar e desativar funcionalidades
Cuidador	Visualizar alertas, configurar lembretes e monitorizar o bem-estar do Idoso

2 Restrições

Nesta secção, identificámos as principais restrições que condicionam o desenvolvimento da arquitetura.

2.1 Restrições Técnicas

RT-01 Dependência da plataforma: Home Assistant (HA)

O sistema integra com o Home Assistant (HA) através das interfaces disponibilizadas pela plataforma. A solução deve:

- utilizar APIs padrão do HA para ler eventos e enviar comandos;
- manter compatibilidade com atualizações do HA, testando e ajustando a integração quando necessário;

RT-02 Restrição de Hardware — Proibição do uso de sensores intrusivos

A recolha de áudio/vídeo é proibida para proteger a privacidade do Idoso.

- **Proibido:** câmaras, microfones, captura contínua de localização pessoalmente identificável.
- **Permitido:** sensores de movimento, abertura/fecho, temperatura, humidade, consumo energético.

RT-03 Limitação de Infraestrutura

A solução deverá combinar de maneira equilibrada a execução local e o uso de serviços cloud:

- **Execução local:** funcionar em dispositivo de baixo custo instalado na habitação, responsável pela recolha de eventos e atuação básica.
- **Conectividade:** conectar-se à rede local para interagir com dispositivos e aceder à Internet para receber notificações e funcionalidades avançadas.
- **Serviços cloud:** utilizar serviços externos quando necessário (por exemplo, para aprendizagem automática ou envio de alertas), garantindo a privacidade e legislação aplicável.

RT-04 Necessidade de acesso à internet

- **Conectividade:** conectar-se à rede local para interagir com dispositivos e aceder à Internet para enviar dados à interface do cuidador.
- **Serviços cloud:** utilizar serviços externos quando necessário (por exemplo, para aprendizagem automática ou envio de alertas), garantindo a privacidade e legislação aplicável.

2.2 Restrições de Negócio

RN-01 Diminuição de Riscos Financeiros

Evitar perdas financeiras decorrentes de incidentes de segurança e descumprimento legal, garantindo a adoção de boas práticas de proteção de dados e conformidade com a legislação.

RN-02 Baixo Custo de Implementação

A solução deve ser financeiramente acessível ao utilizador final, privilegiando:

- hardware de baixo custo e facilmente acessível (objetivo: custo por instalação $\leq$ 150 €);
- custos operacionais reduzidos (objetivo: $\leq$ 10 €/mês por utilizador);
- tecnologias e licenças *open-source*.

3 Contexto e Âmbito

Esta secção estabelece as fronteiras do sistema Guardian e identifica os seus intervenientes de comunicação. A interação com o ecossistema é descrita tanto numa perspetiva funcional (contexto do negócio) como numa perspetiva de infraestrutura (canais e protocolos de comunicação), permitindo compreender de que forma o sistema se articula com o exterior.

3.1 Contexto de Negócio

A tabela seguinte identifica os principais intervenientes que comunicam com o sistema Guardian a nível funcional (atores humanos e sistemas externos), descrevendo os dados ou interações que são trocados entre eles e a plataforma.

Parceiro de Comunicação	Entrada no Sistema	Saída do Sistema
Idoso	Interação com a interface do Idoso; movimento físico detetado por sensores; confirmação de lembretes	Iluminação automatizada; simulação do nascer do sol; lembretes visuais; ajuste da temperatura
Cuidador	Credenciais de autenticação; configuração de lembretes; gestão de associações; consulta do painel	Acesso ao Painel de Tranquilidade; notificações de anomalias; alertas de falhas do sistema
Home Assistant (HA)	Eventos de sensores (movimento, temperatura, humidade, estado da cama); estado dos dispositivos	Comandos de atuação (ligar/desligar luzes, ajustar climatização); configuração de automações
Sensores IoT	Dados de movimento, temperatura, humidade, presença, estado de abertura/fecho	— (não origina saída direta, processamento interno)
Dispositivos Inteligentes	Estado atual (ligado/desligado, intensidade, temperatura)	Comandos de controlo (ligar, desligar)
Serviços Cloud	Dados anonimizados para processamento ML; logs do sistema	Modelos de deteção de anomalias treinados; notificações push
Plugin Manager	Estado de saúde dos componentes; falha de eventos	Comandos de reinício; alertas de recuperação automática
Event Bus (Message Queue)	Eventos de sensores persistidos; comandos de atuação	Eventos distribuídos pelos componentes subscritos; confirmações de entrega

As trocas de informação apresentadas ilustram o fluxo de dados do ponto de vista funcional, permitindo perceber de que modo o sistema responde aos estímulos externos, mantendo sempre o foco na segurança, privacidade e qualidade de vida do idoso.

3.2 Contexto Técnico

A tabela que se segue especifica os protocolos de rede, canais de transmissão e tecnologias utilizadas pelo Guardian para estabelecer comunicação com os elementos externos. É ainda apresentada a correspondência entre os fluxos de dados funcionais (inputs/outputs) e as respetivas infraestruturas de comunicação.

Canal Técnico	Tipo de Comunicação	Mapeamento de Entradas/Saídas
Wi-Fi	Rede local sem fios	Comunicação com dispositivos inteligentes (lâmpadas, termostatos); acesso à interface do Idoso através de API REST
HTTP / HTTPS (REST API)	Comunicação com Home Assistant	Comandos de atuação para dispositivos; leitura de estados e eventos via API REST do HA processados pelo HA Adapter Plugin
WebSocket	Conexão persistente com Home Assistant	Stream de eventos em tempo real distribuídos através do Event Bus para os plugins subscritos
MQTT	Protocolo pub/sub para IoT	Publicação de eventos de sensores no Event Bus; subscrição de comandos de atuação pelos Actuation Plugins
Firebase Cloud Messaging / Apple Push Notification	Notificações push	Alertas de anomalias comportamentais; falhas do sistema detetadas pelo Plugin Manager; confirmação de lembretes
Ethernet (LAN)	Rede local física	Conexão do dispositivo Guardian (Microkernel + Plugins) à rede doméstica e router
Message Queue (Interna)	Event Bus durável com roteamento rápido	Todos os eventos de sensores e comandos circulam através do Event Bus antes de serem processados pelos plugins; persistência em Database para reprocessamento após falhas

Nota sobre a Arquitetura: O Guardian adota uma arquitetura híbrida combinando Microkernel (isolamento total entre plugins) e Event-Driven (comunicação assíncrona via Event Bus). Esta abordagem garante que falhas num plugin nunca comprometem o sistema inteiro, e o Plugin Manager monitoriza continuamente o estado de saúde de todos os componentes, executando recuperação automática quando necessário.

Nota sobre a Privacidade: Todos os dados de sensores brutos permanecem no dispositivo local e são processados pelo Microkernel. Apenas dados agregados e anonimizados são transmitidos para serviços cloud quando estritamente necessário para funcionalidades de aprendizagem automática, cumprindo o princípio de minimização de dados estabelecido nas restrições técnicas (RT-02) e de negócio (RN-01).

4 Estratégia de Solução

4.1 Decisões Tecnológicas

A seleção da stack tecnológica do Guardian foi guiada pelas Restrições Técnicas (RT) e Atributos de Qualidade (AQ) definidos, privilegiando a execução local e a interoperabilidade.

Hardware de Suporte

Para cumprir a restrição de custo (RN-02) e a necessidade de execução local (RT-03), optou-se pelo uso de Raspberry Pi. Esta plataforma oferece o processamento necessário para correr o núcleo do sistema localmente com baixo consumo energético, garantindo operação contínua.

Comunicação entre Módulos

A comunicação interna segue o padrão Publish-Subscribe utilizando o protocolo MQTT. O MQTT garante o desacoplamento entre produtores (sensores) e consumidores (plugins), permitindo a troca de mensagens assíncrona essencial para a arquitetura Event-Driven.

Linguagem e Runtime

O núcleo do sistema é desenvolvido em Python. O Python facilita o desenvolvimento rápido e oferece bibliotecas robustas para automação e Machine Learning, essenciais para os componentes de inteligência do sistema. Para além disso, é uma ferramenta open-source.

Persistência de Dados

A base de dados local escolhida é o SQLite. Sendo serverless e baseada num ficheiro, o SQLite é ideal para ambientes com recursos limitados, assegurando a persistência e privacidade dos dados localmente sem a sobrecarga de um servidor dedicado.

Segurança nas Comunicações

As comunicações na rede local são protegidas por **TLS (Transport Layer Security)**. Garante a integridade e confidencialidade dos dados trocados entre as interfaces e o núcleo do sistema num ambiente de rede doméstico.

4.2 Decisões sobre o Design da Arquitetura

A arquitetura do Guardian adota uma decomposição baseada no padrão **Microkernel** suportada por um modelo de comunicação **Event-Driven**. Esta abordagem separa o sistema em duas partes distintas: um núcleo (Kernel) e módulos de extensão.

O **Kernel** atua como orquestrador, responsável apenas pela gestão do ciclo de vida dos componentes e pelo encaminhamento de mensagens. Ele garante a estabilidade global, isolando falhas.

Os **Plugins** encapsulam a lógica de negócio específica e operam de forma autónoma. Esta separação permite evoluir cada funcionalidade independentemente sem afetar o resto do sistema.

A comunicação entre estes componentes segue um padrão **Publish-Subscribe** através de um **Event Bus** central. Este desacoplamento permite o processamento paralelo e

assíncrono: um evento de sensor é distribuído simultaneamente para múltiplos plugins, garantindo que a atuação crítica ocorre em tempo real.

4.3 Decisões sobre Objetivos de Qualidade

Para atingir os objetivos de qualidade definidos, a arquitetura foi estruturada com as seguintes decisões:

Quality Goal	Estratégia
Availability	Antes de qualquer evento ser processado, é persistido em disco. Isto garante que, em caso de falha de energia ou reinício do sistema, nenhum dado crítico se perde. Complementarmente, o componente <i>Plugin Manager</i> assegura o reinício automático de serviços falhados.
Reliability	A adoção do padrão Microkernel permite isolar a lógica crítica de componentes menos estáveis. Como cada plugin opera de forma autônoma, um erro num módulo não se propaga aos restantes. O <i>Plugin Manager</i> monitoriza a saúde de cada componente e reinicia-os individualmente em caso de falha, garantindo uma operação contínua e sem bugs.
Performance	O sistema utiliza um Event Bus central para distribuir mensagens. Isto permite o processamento paralelo — quando um sensor dispara, o evento é enviado simultaneamente para a atuação e para a análise. Desta forma, a reação física é imediata.
Integrability	O sistema adota o padrão Adapter através do HA Integration Plugin, que encapsula toda a comunicação com a API do Home Assistant (REST e WebSocket). Esta camada de abstração isola o núcleo do Guardian das especificidades da plataforma externa, permitindo que alterações na API do HA sejam absorvidas apenas pelo plugin adaptador, sem impactar os restantes componentes do sistema.
Usability	As interfaces do Idoso e do Cuidador foram desenhadas seguindo princípios de User-Centered Design. A interface do Idoso é simplificada, utilizando elementos visuais de alto contraste, tipografia de grandes dimensões e feedback visual claro para cada ação, minimizando a carga cognitiva e facilitando a interação em tablets.
Security	Implementação de Transport Layer Security (TLS/HTTPS) em todas as comunicações entre as interfaces e o sistema. Além disso, a arquitetura garante a Data Minimization, expondo para a Cloud apenas dados anonimizados estritamente necessários, mantendo os dados sensíveis na base de dados local.
Testability	A arquitetura desacoplada permite testar o núcleo do sistema sem hardware real. É possível substituir o HA Integration Plugin por um Mock Plugin que injeta eventos simulados e permite manipular o relógio do sistema, facilitando a validação de cenários complexos em ambiente de desenvolvimento.

4.4 Processo de Desenvolvimento da Arquitetura

O processo de desenvolvimento da arquitetura foi conduzido de forma sistemática e iterativa, seguindo estas etapas principais:

Definição do Produto e Âmbito

A fase inicial focou-se na identificação de uma oportunidade de mercado viável no contexto de sistemas IoT. Através de sessões de brainstorming, a equipa convergiu para a necessidade de criar uma solução baseada no Home Assistant que endereçasse um problema social relevante. Desta análise surgiu o conceito do Guardian: um sistema de assistência ativa para a população sénior. Nesta etapa refinaram-se as funcionalidades importantes, pensando já na elaboração dos Requisitos Arquiteturais Significativos (ASRs).

Use Cases e Cenários de Qualidade

Com o âmbito definido, procedeu-se à especificação detalhada das interações do sistema. Desenvolveram-se os Use Cases para mapear as interações funcionais entre os atores e o sistema. Paralelamente definiram-se os Cenários de Atributos de Qualidade. Esta atividade permitiu priorizar os requisitos não funcionais estabelecendo métricas concretas de resposta que a arquitetura teria obrigatoriamente de satisfazer.

Desenho da Arquitetura

A terceira fase centrou-se na materialização da solução técnica, iniciando-se com uma análise comparativa de estilos arquiteturais. A equipa selecionou uma abordagem híbrida, combinando o padrão Microkernel com comunicação Event-Driven, por considerar ser a que melhor respondia às necessidades identificadas de extensibilidade e performance. Para documentar a arquitetura, utilizou-se o Modelo C4, permitindo uma decomposição hierárquica e clara do sistema em três níveis distintos: contexto, contentores e componentes.

Resolução de Problemas

O desenvolvimento da arquitetura não foi linear, mas sim um processo de refinamento contínuo. Durante a modelação, foram identificados potenciais pontos de falha e inconsistências entre os requisitos iniciais e a viabilidade técnica. Estas descobertas levaram a ajustes iterativos na arquitetura, garantindo que a solução final fosse robusta, viável e estritamente alinhada com os ASRs definidos.

5 Visão do Contexto (C1)

Neste diagrama de alto nível, temos uma visão geral do funcionamento do sistema Guardian e das suas interações com o mundo exterior, envolvendo os seguintes elementos:

- **Idoso** — O residente monitorizado. Interage passivamente (através de sensores) e ativamente (através de uma interface simplificada em tablet) para gerir automações.
- **Cuidador** — O familiar ou profissional responsável. Utiliza uma aplicação móvel para configurar o sistema e receber alertas de segurança.
- **Guardian System** — O sistema central desenvolvido. Orquestra toda a lógica de segurança, deteta anomalias e comunica com as plataformas externas.
- **Home Assistant** — Plataforma IoT externa existente na casa. É responsável por gerir diretamente o hardware (sensores de movimento, luzes, temperatura) e fornecer esses dados ao Guardian.
- **Serviço de Notificações** — Sistema externo usado para enviar alertas urgentes para o telemóvel do Cuidador quando este não está na aplicação.
- **Serviço de Treino ML** — Serviço externo na Cloud usado esporadicamente para treinar modelos de Inteligência Artificial pesados.

Fluxo de Funcionamento: O Home Assistant recolhe os dados dos sensores espalhados pela casa e envia-os para o Guardian System. Este analisa os dados em tempo real: se detetar uma situação de risco imediato (ex: levantar da cama à noite), envia um comando de volta ao Home Assistant para acender as luzes; se detetar uma anomalia comportamental grave, contacta o Serviço de Notificações para alertar o Cuidador. Paralelamente, o Cuidador pode aceder ao sistema para consultar o histórico e ajustar definições de segurança.

Este diagrama encontra-se representado mais abaixo neste relatório, Figura 2.

6 Visão de Containers (C2)

Neste nível de abstração, identificamos os principais contentores de software que compõem o ecossistema Guardian, detalhando as suas responsabilidades e tecnologias de implementação. Esta decomposição reflete a estratégia de execução híbrida:

Interfaces de Utilizador

- **Elderly Interface:** Aplicação Web Progressiva desenhada especificamente para tablets fixos na residência. Oferece uma interface simplificada e de alto contraste (AQ-05 Usability) que permite ao Idoso visualizar lembretes (UC-03) e gerir preferências de automação (UC-06). Comunica exclusivamente com o Core via HTTPS.
- **Caregiver Interface (Mobile):** Aplicação nativa para o smartphone do Cuidador. Funciona como o painel de controlo remoto (UC-07), permitindo a receção de notificações Push críticas (UC-05) e a configuração das regras de segurança (UC-09).

Persistência de Dados

Local Database: Contentor da base de dados relacional que reside no mesmo ambiente de execução do Core. Armazena todos os dados sensíveis, incluindo perfis de utilizador, configurações de automação e o histórico dos eventos de sensores. A escolha de uma base de dados local, não exposta à Internet, é fundamental para garantir o requisito da Privacidade (AQ-06 Security).

Interação com o Núcleo

Tanto a Elderly Interface como a Caregiver Interface interagem com o sistema central através do Guardian Core, que expõe uma API segura. Nenhuma interface acede diretamente à Base de Dados ou ao Home Assistant, garantindo o desacoplamento e a segurança da arquitetura.

Este diagrama encontra-se representado mais abaixo neste relatório, Figura 3.

7 Visão de Componentes (C3)

Neste diagrama de componentes, detalhamos o funcionamento interno do contentor "Guardian Core", ilustrando a separação entre o núcleo de gestão e os módulos funcionais:

Guardian Host (Kernel)

A infraestrutura central que garante a estabilidade e orquestração:

- **Event Bus** — O canal de comunicação central do tipo Publish-Subscribe. Distribui mensagens entre plugins de forma assíncrona, permitindo o processamento paralelo.
- **Plugin Manager** — O componente "Plugin Manager" responsável por carregar, monitorizar e reiniciar automaticamente qualquer plugin que falhe.
- **Scheduler Service** — Serviço responsável pela gestão de eventos temporais, como o Despertador e Lembretes.

Plugins (Extensões)

Módulos independentes que contêm a lógica do negócio específica:

- **HA Integration Plugin** — O adaptador que isola a comunicação com o Home Assistant, protegendo o sistema de alterações externas.
- **Safety & Climate Plugin** — Contém a lógica crítica da atuação imediata (ex: acender luzes, gerir temperatura).
- **ML Plugin** — Módulo de inteligência que analisa padrões de comportamento a longo prazo e comunica com a Cloud para o treino de modelos.
- **API Gateway Plugin** — Ponto de entrada seguro para as aplicações móveis, responsável pela autenticação e filtragem de dados.
- **Persistence Plugin** — Responsável pela escrita do histórico de eventos na base de dados local.

Fluxo de Funcionamento

Os dados dos sensores entram pelo HA Integration Plugin e são imediatamente enviados para o Event Bus que posteriormente são escritos em memória. De seguida, são publicados e distribuídos simultaneamente: o Safety Plugin processa-os para uma reação imediata (atuando sobre as luzes via HA Plugin), enquanto o Anomaly Detection Plugin os analisa em busca de desvios comportamentais. Todo este ciclo é vigiado pelo Plugin Manager para garantir operação contínua.

Este diagrama encontra-se representado mais abaixo neste relatório, Figura 4.

8 Visão de Deployment

Esta secção apresenta a arquitetura de deployment do sistema Guardian, descrevendo como os componentes são fisicamente distribuídos e executados na infraestrutura real.

8.1 Estratégia de Deployment

O sistema Guardian é implantado em dois ambientes distintos:

Ambiente Local — Residência do Idoso

O núcleo do sistema executa localmente num **Raspberry Pi 3/4**, garantindo baixa latência nas respostas críticas e proteção de dados sensíveis. O dispositivo opera continuamente com o seguinte setup:

- **Docker Engine:** Todos os componentes executam em contentores Docker isolados, facilitando atualizações e recuperação de falhas
- **Guardian Core Container:** Contentor principal com Event Bus (MQTT), Plugin Manager, e todos os plugins (HA Integration, Safety & Climate, ML, API Gateway, Persistence, Scheduler)
- **Elderly Interface Container:** Servidor Nginx que serve a PWA para o tablet do idoso
- **Home Assistant Container:** Executa separadamente, gerindo os dispositivos IoT físicos
- **SQLite Database:** Base de dados embebida no Guardian Core Container

O tablet do idoso acede à interface através de um navegador web na rede local via Wi-Fi.

Ambiente Cloud — Infraestrutura Externa

Serviços complementares executam na cloud (AWS/Azure/GCP):

- **Caregiver Mobile App:** Aplicação nativa distribuída pelas app stores
- **Push Notification Services:** Firebase/Apple Push para alertas ao cuidador
- **ML Training Service:** Processamento batch esporádico de modelos de ML com dados anonimizados

8.2 Requisitos de Infraestrutura

A tabela seguinte apresenta o custo estimado dos componentes necessários para uma instalação completa:

Componente	Custo (€)	Observações
Raspberry Pi 4 (4GB RAM)	60	Inclui fonte de alimentação
Cartão SD 32GB (Classe A2)	10	Armazenamento do sistema
Case com ventilação	15	Dissipação de calor
Cabo Ethernet CAT6 (5m)	5	Ligação ao router
Tablet Android (usado)	50	Interface do idoso
Total por instalação 140		
Cumpre restrição RN-02 (150€)		

Nota: Dispositivos IoT (sensores, lâmpadas) e router não incluídos, assumindo instalação Home Assistant pré-existente.

8.3 Diagrama de Deployment

A Figura 5 ilustra a distribuição física dos componentes, mostrando a topologia de deployment do sistema Guardian numa residência típica com execução em Raspberry Pi e containers Docker.

9 Conceitos Transversais

Para além da decomposição modular em plugins, a arquitetura do Guardian implementa mecanismos transversais que garantem a coesão, segurança e operacionalidade do sistema. Estes conceitos atravessam as fronteiras dos componentes individuais para responder aos Atributos de Qualidade globais.

9.1 Monitorização e Tolerância a Falhas

Para satisfazer os requisitos críticos de Safety (AQ-01) e Reliability, o sistema implementa um padrão de **Watchdog** centralizado no componente Plugin Manager. Este mecanismo monitoriza continuamente o estado de saúde de todos os plugins. Em caso de bloqueio ou falha de um módulo, o gestor reinicia-o automaticamente, garantindo que o sistema recupera para um estado operacional, sem intervenção humana.

9.2 Segurança e Privacidade

A segurança é tratada como um aspeto transversal aplicado em todas as camadas, respondendo ao AQ-06:

- **Comunicação Segura:** Todas as trocas de dados entre as interfaces e o núcleo são encriptadas via TLS, protegendo contra interceção na rede local.
- **Minimização de Dados:** A arquitetura impõe que os dados brutos dos sensores residam estritamente na base de dados local. A comunicação com a Cloud é restrita a dados anonimizados para treino de ML ou notificações de alerta, garantindo a privacidade do idoso por desenho.

10 Decisões Arquiteturais

Esta secção documenta as principais decisões arquiteturais tomadas durante o desenvolvimento do Guardian, apresentando os prós e contras de cada abordagem, bem como a justificação para as escolhas realizadas.

10.1 DA-01: Adoção do Estilo Arquitetural Microkernel + Event-Driven Contexto e Motivação

A adoção de uma arquitetura Microkernel combinada com um modelo Event-Driven foi motivada pelas necessidades específicas do sistema Guardian, nomeadamente a monitorização contínua, a execução de automações sensíveis ao contexto e a integração com a plataforma Home Assistant (HA). Dado que o Guardian opera num ambiente de automação orientado à segurança e bem-estar do idoso, tornou-se fundamental garantir que o componente central do sistema permanece estável, confiável e facilmente escalável, mesmo perante alterações externas ou falhas pontuais.

A abordagem Microkernel permite que o núcleo do Guardian se mantenha focado nas responsabilidades essenciais, como a gestão de eventos, coordenação de automações e aplicação das regras centrais de decisão (por exemplo, nos use cases UC-01 a UC-05). Funcionalidades específicas, como a deteção de anomalias comportamentais, a gestão de lembretes, o controlo climático ou o adaptador de integração do Home Assistant, são implementadas como plugins independentes, o que facilita a manutenção, a testabilidade (AQ-07) e a evolução incremental do sistema sem comprometer a sua disponibilidade (AQ-02) ou segurança das operações (AQ-01).

Complementarmente, o modelo Event-Driven adequa-se de forma natural ao funcionamento do Guardian, uma vez que o sistema reage continuamente a eventos provenientes de sensores e a ações dos utilizadores. A comunicação assíncrona reduz o nível de interdependência entre componentes, permite processamento paralelo e melhora a escalabilidade, assegurando tempos de resposta competentes e compatíveis com os requisitos de desempenho definidos (AQ-03). Além disso, esta abordagem aumenta a tolerância a falhas, permitindo a persistência e o reproprocessamento de eventos críticos em cenários de indisponibilidade temporária.

Esta combinação arquitetural responde diretamente aos principais atributos de qualidade do Guardian, confiabilidade, disponibilidade, desempenho, integrabilidade e testabilidade, e suporta de forma eficaz a restrição técnica de dependência do Home Assistant (RT-01), isolando as mudanças externas no respetivo adaptador. Assim, a arquitetura escolhida fornece uma base sólida, flexível e resiliente para um sistema distribuído e orientado a eventos, adequado a ambientes de automação com foco na proteção e qualidade de vida do idoso.

Prós

- **Isolamento total de funcionalidades:** Cada plugin opera de forma independente, garantindo que a falha de um módulo não compromete o sistema como um todo.
- **Extensibilidade facilitada:** Novos plugins podem ser adicionados sem modificar o núcleo do sistema, permitindo evolução e implementação de novas funcionalidades.
- **Manutenção simplificada:** Componentes isolados podem ser atualizados, testados e corrigidos individualmente.

- **Baixa dependência entre componentes:** A comunicação assíncrona via Event Bus reduz dependências diretas entre componentes.
- **Robustez operacional:** O Plugin Manager monitoriza continuamente o estado dos componentes e efetua reinicializações automáticas caso necessário.
- **Processamento paralelo:** Múltiplos plugins podem processar eventos simultaneamente, otimizando o desempenho.
- **Adaptabilidade à plataforma Home Assistant:** O núcleo mantém-se estável enquanto adaptadores específicos (plugins) lidam com as particularidades da integração externa da plataforma Home Assistant.

Contras

- **Complexidade de coordenação:** A comunicação assíncrona pode tornar o debugging mais desafiante, especialmente em fluxos que envolvem múltiplos plugins.
- **Overhead de mensagens:** O Event Bus introduz latência adicional na comunicação entre componentes.
- **Gestão da distribuição do estado do sistema:** O estado do sistema encontra-se fragmentado entre plugins, exigindo mecanismos robustos de sincronização.
- **Potencial de duplicação de lógica:** Funcionalidades semelhantes podem ser implementadas mais que uma vez em diferentes plugins.
- **Dependência crítica do Plugin Manager:** Embora isolado, o Plugin Manager torna-se um ponto central cuja falha pode afetar a recuperação automática do sistema.

10.2 DA-02: Escolha de Python como Linguagem Principal

Contexto e Motivação

A escolha da linguagem de programação tem impacto direto no desempenho do sistema e na capacidade de integração com ecossistemas externos.

Prós

- **Ecossistema rico para ML:** Bibliotecas como scikit-learn, TensorFlow e PyTorch facilitam a implementação do componente de deteção de anomalias.
- **Integração com Home Assistant:** O HA é desenvolvido em Python, facilitando a compreensão da API e a resolução de problemas de integração.
- **Produtividade no desenvolvimento:** Sintaxe clara e bibliotecas abundantes aceleram o desenvolvimento do projeto e a iteração rápida.
- **Compatibilidade com Raspberry Pi:** Suporte nativo e otimizado para ARM, alinhado com a restrição de hardware (RT-03).

Contras

- **Desempenho inferior:** Python é interpretado, resultando num maior consumo de CPU e memória em comparação com linguagens que requerem compilação.
- **Gestão de concorrência:** Problemas relacionados com operações CPU-intensive.

Mitigação dos Contrás

Para mitigar as limitações no desempenho:

- Nas operações críticas de baixa latência (ex: processamento do "Caminho Seguro") o ideal é utilizar bibliotecas otimizadas em C (NumPy, asyncio).
- As tarefas CPU-intensive (treino de ML) são atribuídas a serviços cloud quando necessário.

10.3 DA-03: SQLite como Base de Dados Local

Contexto e Motivação

A escolha do sistema de gestão da base de dados local deve equilibrar desempenho, consumo de recursos e conformidade com requisitos de privacidade.

Prós

- **Zero configuração:** Base de dados serverless baseada num ficheiro, ideal para Edge Computing (RT-03).
- **Footprint mínimo:** Baixo consumo de memória e CPU, adequado ao Raspberry Pi.

Contrás

- **Limitações de concorrência:** Escritas concorrentes podem causar bloqueios.
- **Ausência de replicação nativa:** Backup requer implementação manual.

Mitigação dos Contrás

- Sistema de backup automático diário para o serviço cloud (opcional, mediante consentimento do utilizador).

10.4 DA-04: MQTT como Protocolo de Comunicação Interna

Contexto e Motivação

A comunicação entre plugins e o Event Bus central requer um protocolo leve, confiável e adequado a ambientes de recursos limitados.

Prós

- **Padrão Publish-Subscribe nativo:** Alinhado com a arquitetura Event-Driven escolhida.
- **Overhead mínimo:** Protocolo leve ideal para dispositivos IoT e Edge Computing.
- **Qualidade de Serviço configurável:** Permite garantir a entrega de mensagens críticas e otimizar latência para eventos menos críticos.

Contras

- **Ausência de schemas:** Falta de validação da estrutura das mensagens pode causar erros de integração.
- **Complexidade de debug:** Fluxos assíncronos dificultam o rastreamento de bugs (já identificado como contra da arquitetura).

Mitigação dos Contras

- Definição de schemas JSON para eventos críticos, com validação obrigatória nos plugins.
- Implementação de distributed tracing com identificadores de correlação em todas as mensagens.

10.5 DA-05: PWA para Interface do Idoso

Contexto e Motivação

A interface do Idoso (UC-06) deve ser acessível e simples, e funcionar em tablets fixos na residência sem necessidade de instalação de apps nativas.

Prós

- **Sem necessidade de app store:** Instalação via browser reduz desistências e custos de manutenção.
- **Atualizações transparentes:** Nova versão disponível imediatamente sem ação do utilizador.
- **Funcionamento offline:** Service Workers permitem operações básicas mesmo sem Internet.
- **Multiplataforma:** Funciona em qualquer tablet moderno (Android, iOS, Windows).

Contras

- **Limitações da API:** Acesso restrito a funcionalidades nativas do dispositivo comparado a apps nativas.
- **Suporte inconsistente:** Algumas funcionalidades PWA variam entre browsers.

Justificação

Os contras são aceitáveis porque a interface do Idoso não requer funcionalidades nativas avançadas (não usa câmara, GPS, etc.). A simplicidade do deployment e atualização é prioritária para garantir Usability (AQ-05) e reduzir custos operacionais (RN-02).

10.6 Tabela com o Resumo das Decisões Arquiteturais

ID	Decisão	Justificação Principal	Atributos de Qualidade
DA-01	Microkernel + Event-Driven	Isolamento de falhas e processamento paralelo	AQ-01, AQ-02, AQ-03, AQ-04, AQ-07
DA-02	Python como linguagem principal	Ecossistema ML e integração com HA	AQ-04, AQ-07
DA-03	SQLite como BD local	Privacidade e zero configuração	AQ-06, RT-03, RN-02
DA-04	MQTT como protocolo interno	Comunicação assíncrona leve	AQ-02, AQ-03, RT-03
DA-05	PWA para interface do Idoso	Simplicidade do deployment	AQ-05, RN-02

11 Atributos de Qualidade

AQ-01 Reliability

Descrição

A capacidade do sistema manter os seus serviços operacionais e recuperar de falhas internas num tempo aceitável, garantindo a monitorização contínua do Idoso.

Cenário: Falha na execução do "Caminho Seguro"(UC-01)

- **Fonte:** Componente interno.
- **Estímulo:** O sistema falha exatamente no momento em que o Idoso se levanta da cama (UC-01).
- **Ambiente:** Operação normal, durante a noite.
- **Artefacto:** O sistema.
- **Resposta:** Um sistema de deteção de falhas deve detetar a avaria, reiniciar automaticamente o sistema e notificar o Cuidador.
- **Medida:** Serviço falhado é reiniciado automaticamente em < 5 segundos; o "Painel de Tranquilidade"(UC-07) é atualizado para o estado "Atenção: Falha no sistema de automação"em < 2 minutos após a falha.

AQ-02 Availability (Disponibilidade)

Descrição

A proporção de tempo em que o sistema está operacional e a executar as suas funções, especialmente os serviços de monitorização (UC-05).

Cenário: Falha na Deteção de Anomalias (UC-05)

- **Fonte:** Componente interno.
- **Estímulo:** O serviço de Deteção de Anomalias (UC-05) falha inesperadamente.
- **Ambiente:** Operação normal, 24/7.
- **Artefacto:** O serviço de Deteção de Anomalias.
- **Resposta:** O sistema deve reiniciar automaticamente o serviço falhado. Para minimizar perdas, os eventos dos sensores que ocorrerem durante a falha devem ser persistidos em memória e processados assim que o serviço reiniciar.
- **Medida:** Serviço reiniciado automaticamente em < 1 minuto; taxa de entrega de eventos $\geq 99,9\%$ durante falhas.

AQ-03 Performance (Desempenho)

Descrição

A capacidade do sistema responder a eventos dentro de limites de tempo aceitáveis. No "Guardian", existem requisitos de latência distintos.

Cenário: Latência do "Caminho Seguro"(UC-01)

- **Fonte:** Idoso.
- **Estímulo:** O Idoso levanta-se da cama, o sensor envia o evento "movimento"ou "cama vazia".
- **Ambiente:** Operação normal, durante a noite.
- **Artefacto:** Sistema (fim-a-fim: do evento de sensor ao acendimento da luz).
- **Resposta:** O Sistema Guardian deve receber o evento, processar a regra e acionar a luz.
- **Medida:** Latência < 500 ms (do evento de sensor ao acender da luz).

AQ-04 Integrability (Integração)

Descrição

A facilidade com que o sistema se liga e troca dados com sistemas externos, neste caso, o Home Assistant (HA).

Cenário: Alteração da API do Home Assistant

- **Fonte:** Externo (Equipa de desenvolvimento do Home Assistant).
- **Estímulo:** Uma nova versão do HA é lançada, alterando o formato dos eventos recolhidos pelos sensores (uma *breaking change* na API).
- **Ambiente:** Manutenção / Evolução.
- **Artefacto:** O "Adaptador do HA"(o componente de integração do Guardian).
- **Resposta:** A arquitetura deve isolar o conhecimento do HA. O núcleo do sistema (Motor de Regras, Motor de ML) não deve ser afetado. As alterações devem ser contidas apenas no módulo "Adaptador"que traduz os eventos do HA para o modelo de dados interno do Guardian.
- **Medida:** Apenas 1 componente/módulo (o Adaptador) necessita de modificação; zero alterações nos serviços de lógica do negócio.

AQ-05 Usability (Usabilidade)

Descrição

A facilidade com que os atores (Idoso e Cuidador) interagem com o sistema.

Cenário: Idoso desativa uma automação (UC-06)

- **Fonte:** Idoso.
- **Estímulo:** O Idoso decide que não gosta do "Despertador"(UC-02) e quer desativá-lo, mas quer manter o "Caminho Seguro"(UC-01).
- **Ambiente:** Operação normal.

- **Artefacto:** Interface do Idoso (UC-06).
- **Resposta:** O Idoso deve ser capaz de abrir a interface, identificar claramente a automação "Despertador" e desativá-la, com feedback de confirmação.
- **Medida:** Taxa de sucesso da tarefa $\geq 95\%$ num teste de usabilidade com idosos; tempo mediano para completar a tarefa ≤ 10 segundos; critérios mínimos de contraste e tamanho de letra respeitados.

AQ-06 Security (Segurança de Dados)

Descrição

A capacidade de proteger os dados sensíveis do Idoso contra acesso não autorizado, garantindo o pilar de privacidade do produto.

Cenário: Cuidador tenta aceder a dados brutos (UC-07)

- **Fonte:** Utilizador (Cuidador).
- **Estímulo:** Um Cuidador tenta inspecionar as chamadas de rede (ex: via *browser dev tools*) da sua aplicação (UC-07) para ver o histórico exato dos movimentos do Idoso.
- **Ambiente:** Operação normal.
- **Artefacto:** A API do "Painel de Tranquilidade".
- **Resposta:** A API nunca deve expor dados de eventos individuais (exceto o da "última atividade"). A API deve apenas fornecer o estado abstrato ("Tudo OK", "Alerta Ativo") e o histórico de alertas gerados (UC-05).
- **Medida:** Nenhum endpoint disponível para o perfil "Cuidador" expõe dados de sensores individuais, confirmado por testes automatizados de verificação de endpoints e revisão do código de segurança.

AQ-07 Testability (Testabilidade)

Descrição

A facilidade com que o sistema pode ser testado para validar o seu funcionamento.

Cenário: Testar o Alerta de Anomalia (UC-05)

- **Fonte:** Desenvolvedor.
- **Estímulo:** A equipa de QA precisa de validar que o alerta de "Inatividade Prolongada" (UC-05) funciona, sem ter de esperar 4 horas num ambiente real.
- **Ambiente:** Ambiente de Teste.
- **Artefacto:** O sistema.
- **Resposta:** A arquitetura deve permitir a "injeção de eventos" e a utilização do relógio modificável. O *tester* deve poder submeter uma sequência de eventos de sensor falsos e, se necessário, "avançar o relógio" do sistema.

- **Medida:** O sistema deve detetar a anomalia injetada e disparar o alerta em < 1 minuto (em tempo de sistema simulado) após a condição ser cumprida.

12 Riscos e Dívida Técnica

12.1 Riscos Identificados

R-01: Falhas na Detecção de Anomalias

- **Descrição:** O modelo de machine learning pode gerar falsos positivos (alertas desnecessários) ou falsos negativos (não detetar situações críticas), afetando a confiança do sistema.
- **Probabilidade:** Média | **Impacto:** Alto
- **Mitigação:** Período de aprendizagem inicial obrigatório de pelo menos 1 semana. Implementação de limiares ajustáveis e validação contínua do modelo. Sistema de feedback que permite ao Cuidador classificar os alertas recebidos, ajustando o modelo ao longo do tempo.

R-02: Indisponibilidade dos Serviços Cloud

- **Descrição:** Falhas em serviços externos (notificações push, processamento de ML avançado) podem comprometer funcionalidades críticas como alertas ao Cuidador.
- **Probabilidade:** Baixa | **Impacto:** Alto
- **Mitigação:** Implementação de mecanismos de fallback locais para funcionalidades críticas. Utilização do armazenamento local para garantir a entrega das notificações após a recuperação do serviço. Cache local com as configurações essenciais.

R-03: Exposição de Dados Sensíveis

- **Descrição:** Dados comportamentais do Idoso podem ser expostos através de vulnerabilidades na API ou configurações inadequadas de permissões.
- **Probabilidade:** Baixa | **Impacto:** Crítico
- **Mitigação:** Implementação de autenticação robusta (UC-08), controlo de acesso baseado em roles (RBAC), protocolo criptográfico que garante a segurança das comunicações (TLS). Exposição apenas de dados sensíveis ao Cuidador, conforme definido em AQ-06.

R-04: Ataques à Infraestrutura Local

- **Descrição:** Dispositivos IoT e o hub local podem ser comprometidos através da rede doméstica, permitindo acesso não autorizado ou manipulação de automações.
- **Probabilidade:** Baixa | **Impacto:** Alto
- **Mitigação:** Segmentação de rede (VLAN dedicada para IoT), atualizações de segurança automáticas, desativação de serviços desnecessários, e monitorização de tentativas de acesso não autorizado.

R-05: Falha do Plugin Manager

- **Descrição:** O Plugin Manager, sendo o componente central de monitorização e recuperação automática, representa um ponto único de falha (single point of failure). Se este componente falhar, o sistema perde a capacidade de detetar e reiniciar plugins falhados automaticamente, comprometendo a fiabilidade global.
- **Probabilidade:** Baixa | **Impacto:** Crítico
- **Mitigação:** Implementação de um watchdog externo ao nível do sistema operativo (systemd) que monitoriza o próprio Plugin Manager e o reinicia em caso de falha. Manter o Plugin Manager com código minimalista e altamente testado para reduzir a probabilidade de bugs. Implementação de logs detalhados que permitam um diagnóstico rápido em caso de falha.

R-06: Rejeição pelos Utilizadores

- **Descrição:** Idosos podem sentir-se desconfortáveis com a monitorização ou ter dificuldades na utilização da interface, resultando em baixa adoção.
- **Probabilidade:** Média | **Impacto:** Alto
- **Mitigação:** Design centrado no utilizador com testes de usabilidade (AQ-05), transparência total sobre os dados recolhidos, controlo total do Idoso sobre automações (UC-06), e programa de onboarding assistido.

R-07: Custos Operacionais Superiores ao Previsto

- **Descrição:** Custos em serviços cloud, suporte técnico ou manutenção podem exceder o limite de 10€/mês por utilizador, comprometendo a viabilidade económica.
- **Probabilidade:** Média | **Impacto:** Médio
- **Mitigação:** Monitorização rigorosa de custos, otimização do uso de recursos cloud, implementação de tiers de serviço, e renegociação de contratos com fornecedores à medida que a base de utilizadores cresce.

12.2 Dívida Técnica

12.2.1 Dívida Técnica Intencional

DT-01: Simplicidade Inicial do Modelo de ML

- **Descrição:** Na primeira versão, o sistema de deteção de anomalias utiliza algoritmos simples (ex: deteção de desvios estatísticos em relação à média) em vez de modelos sofisticados de deep learning.
- **Justificação:** Permite validação rápida do conceito, reduz requisitos computacionais e facilita a interpretabilidade dos resultados.
- **Plano de Resolução:** Após validação em ambiente real e recolha de dados suficientes (6-12 meses), avaliar a implementação de modelos mais avançados (LSTM, Autoencoders) que podem capturar padrões temporais complexos.

DT-02: Gestão de Prioridades de Eventos (Quality of Service)

- **Descrição:** A arquitetura é totalmente orientada a eventos (Event-Driven), processando todas as mensagens de forma assíncrona. Atualmente, todos os eventos são tratados com a mesma prioridade e em casos de tráfego intenso, eventos importantes vêm com latência.
- **Justificação:** Esta abordagem simplificou a implementação inicial do Event Bus e dos Plugins, sendo suficiente para garantir a latência de < 500ms em condições de carga normal na rede doméstica.
- **Plano de Resolução:** Implementar níveis de Quality of Service (QoS) e Filas de Prioridade no broker MQTT..

12.3 Oportunidades de Melhoria

12.3.1 Melhorias de Arquitetura

OM-01: Observabilidade Avançada

- **Descrição:** Implementar stack completo de observabilidade (métricas, logs, traces distribuídos) com ferramentas como Prometheus e Grafana, complementado com Jaeger para debug de fluxos complexos.
- **Benefício:** Facilita debugging de problemas em produção, permitindo a otimização proativa do desempenho.
- **Esforço Estimado:** Médio (3-4 sprints)

12.3.2 Experiência do Utilizador

OM-02: Dashboard Preditivo para Cuidador

- **Descrição:** Evoluir o "Painel de Tranquilidade"(UC-07) para incluir visualizações de tendências e previsões sobre o bem-estar do Idoso.
- **Benefício:** Cuidador pode antecipar necessidades e planejar visitas/intervenções de forma mais eficaz.
- **Esforço Estimado:** Médio (4-5 sprints)

OM-03: Modo de Personalização Assistida

- **Descrição:** Wizard interativo que guia o Idoso e o Cuidador através da configuração inicial, testando os sensores e a automação em tempo real.
- **Benefício:** A redução de barreiras no processo melhora a experiência do utilizador e aumenta a eficiência do fluxo, aumentando a taxa de adoção e a satisfação inicial.
- **Esforço Estimado:** Médio (3-4 sprints)

OM-04: Relatórios Periódicos Automatizados

- **Descrição:** Geração automática de relatórios semanais/mensais sobre as atividades e o bem-estar do Idoso, enviados ao Cuidador.
- **Benefício:** Mantém o Cuidador informado sem necessidade de consulta ativa constante, reduzindo a sua ansiedade.
- **Esforço Estimado:** Baixo (2 sprints)

12.3.3 Conformidade e Segurança

OM-05: Penetration Testing Regular

- **Descrição:** Estabelecer um programa trimestral de testes de penetração por entidade externa certificada.
- **Benefício:** Identificação proativa de vulnerabilidades, reduz o risco de incidentes de segurança (R-03, R-04).
- **Esforço Estimado:** Baixo

13 Glossário

Termo	Definição
API	Application Programming Interface — Interface de programação que permite a comunicação entre sistemas.
ASR	Architecturally Significant Requirement — Requisito que tem impacto significativo na arquitetura do sistema.
C4 Model	Framework de modelação de arquitetura que documenta sistemas em quatro níveis: Context (C1), Container (C2), Component (C3) e Code (C4).
Data Minimization	Princípio de privacidade que estabelece que apenas dados essenciais e estritamente necessários devem ser recolhidos, armazenados e processados.
Durable Queue	Fila de mensagens persistida em disco que garante a entrega e reprocessamento de eventos mesmo após falhas de energia ou reinicializações do sistema.
Edge Computing	Processamento de dados realizado localmente, próximo da fonte dos dados, em vez de na cloud.
Event Bus	Canal central de comunicação assíncrona que distribui eventos entre componentes segundo o padrão Publish-Subscribe.
Event-Driven	Padrão arquitetural onde o fluxo do sistema é determinado por eventos.
Home Assistant (HA)	Plataforma open-source de automação residencial usada como base de integração IoT.
IoT	Internet of Things — Rede de dispositivos físicos conectados à Internet.
Microkernel	Padrão arquitetural que separa funcionalidades mínimas do núcleo em plugins extensíveis.
ML	Machine Learning — Aprendizagem automática.
MQTT	Message Queuing Telemetry Transport — Protocolo de mensagens leve para IoT.
Plugin Manager	Componente de monitorização centralizada que supervisiona o estado de saúde de todos os plugins e executa reinicializações automáticas em caso de falha.
Quality Attribute Scenario	Estrutura de teste que descreve um estímulo, um ambiente, um componente afetado, a resposta esperada e a métrica de sucesso para validar um atributo de qualidade.
REST API	Representational State Transfer — Padrão arquitetural para construir APIs que utiliza operações HTTP (GET, POST, PUT, DELETE) sobre recursos.

Termo	Definição
Plugin	Módulo de software que adiciona funcionalidades específicas a um sistema base.
Publish-Subscribe	Padrão de comunicação onde os produtores publicam mensagens e os consumidores subscrevem tópicos de interesse.
PWA	Progressive Web App — Aplicação web que oferece experiência similar a aplicações nativas.
RBAC	Role-Based Access Control — Controlo de acesso baseado em papéis/funções.
TLS	Transport Layer Security — Protocolo criptográfico para comunicações seguras.
Use Case	Descrição de uma sequência de interações entre um ator externo e um sistema para atingir um objetivo específico, documentando fluxos principais, excecionais e pré/pós-condições.
Watchdog	Componente que monitoriza o estado de outros componentes e reinicia-os em caso de falha.

14 Conclusão

O Guardian apresenta uma solução focada no bem-estar do idoso, combinando monitorização passiva e intervenção proativa, para aumentar a segurança e o conforto no seu quotidiano. Os Use Cases identificados abrangem todo o ciclo de interação entre o Idoso e o Cuidador, enquanto os AQs definem objetivos concretos de confiabilidade, disponibilidade, desempenho, integração, usabilidade, segurança de dados e testabilidade.

As restrições técnicas, como a integração com o Home Assistant, a proibição de sensores intrusivos e a execução local com apoio em serviços de cloud, juntamente com as restrições de negócio, centradas na acessibilidade económica e conformidade legal, estabelecem as bases para uma implementação viável e ética.

A arquitetura proposta, baseada no padrão Microkernel com comunicação Event-Driven, oferece a flexibilidade necessária para evoluir o sistema de forma incremental, mantendo a estabilidade do núcleo. A decomposição em plugins permite isolar falhas e adicionar novas funcionalidades sem comprometer a operação existente.

Os diagramas C4 apresentados documentam a arquitetura em diferentes níveis de abstração, desde o contexto de sistema até aos componentes internos, facilitando a comunicação entre stakeholders técnicos e não técnicos.

A identificação proativa de riscos e dívida técnica, bem como as oportunidades de melhoria documentadas, garantem que a equipa mantém uma visão clara das limitações atuais e do caminho de evolução do sistema.

Este documento consolida os princípios funcionais e arquiteturais que orientam o desenvolvimento do Guardian, fornecendo uma base sólida para a sua implementação e futura evolução.

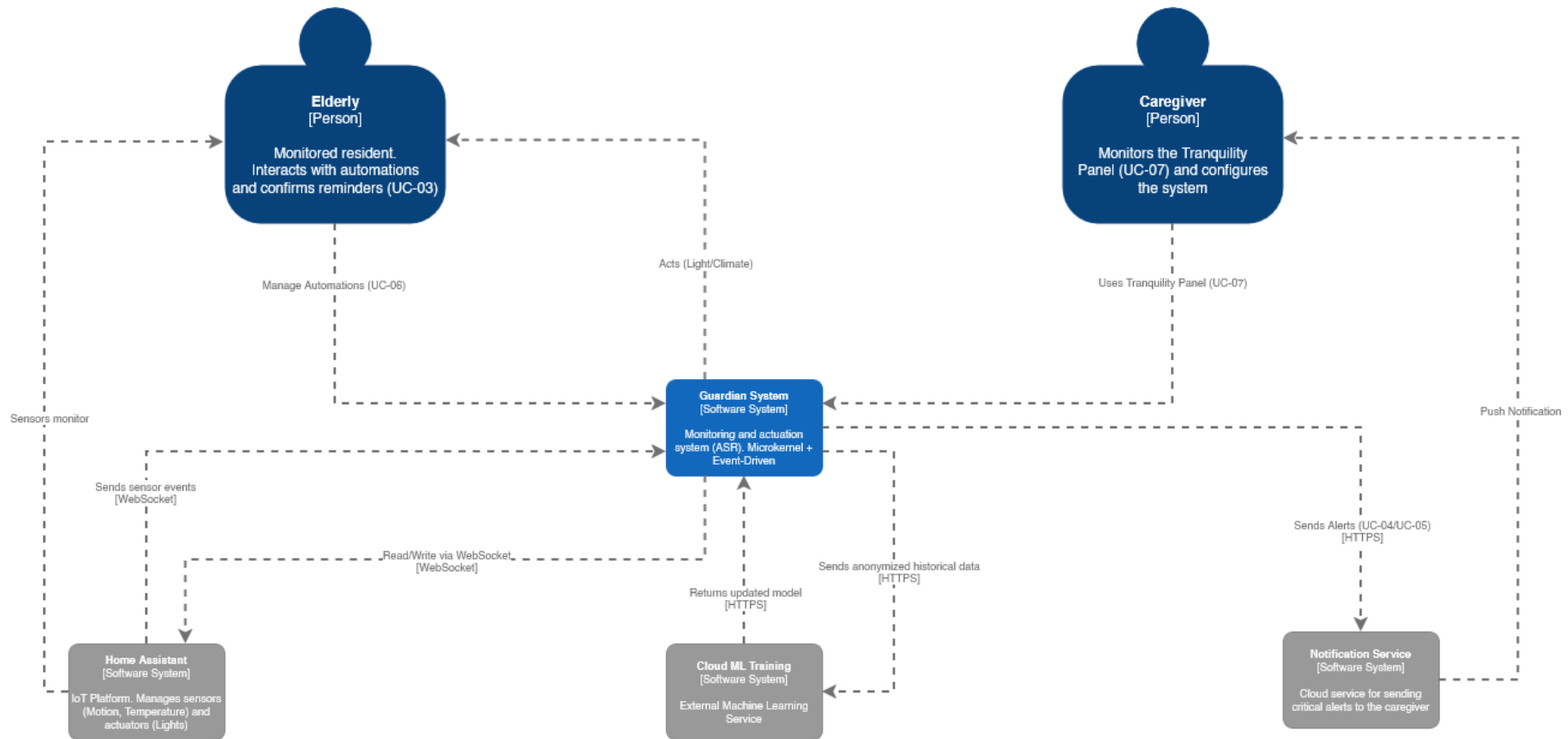


Figura 2: Visão do Contexto do Sistema (C1)

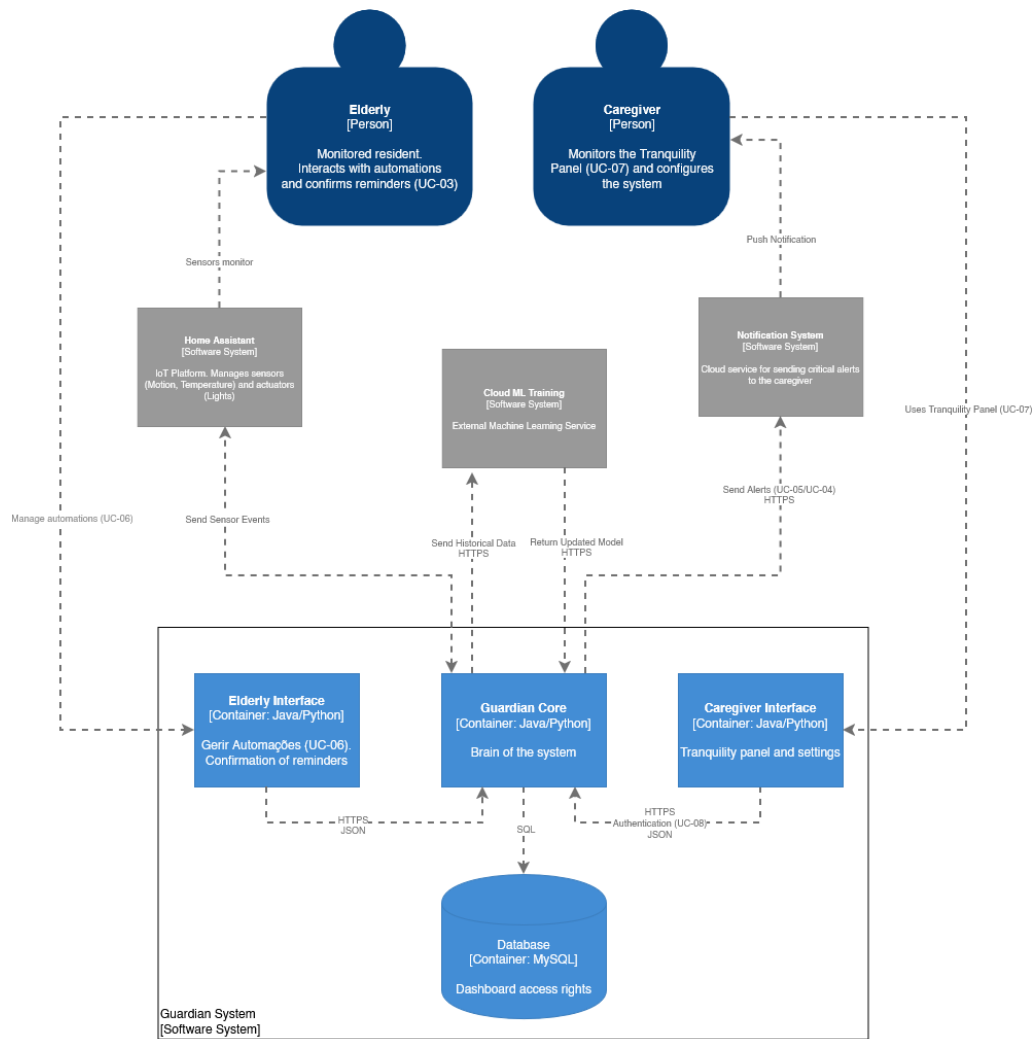


Figura 3: Visão do Container do Sistema (C2)

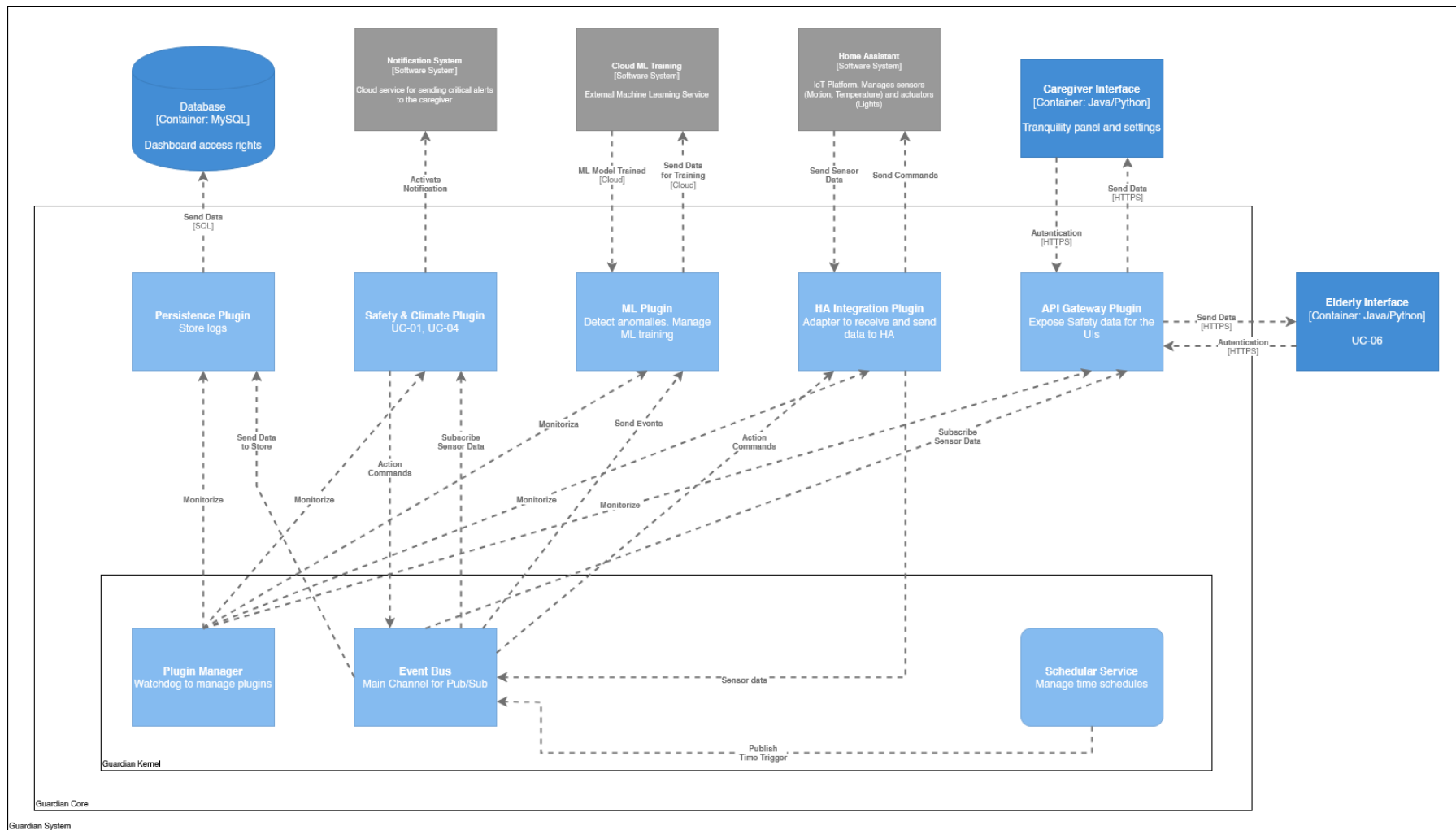
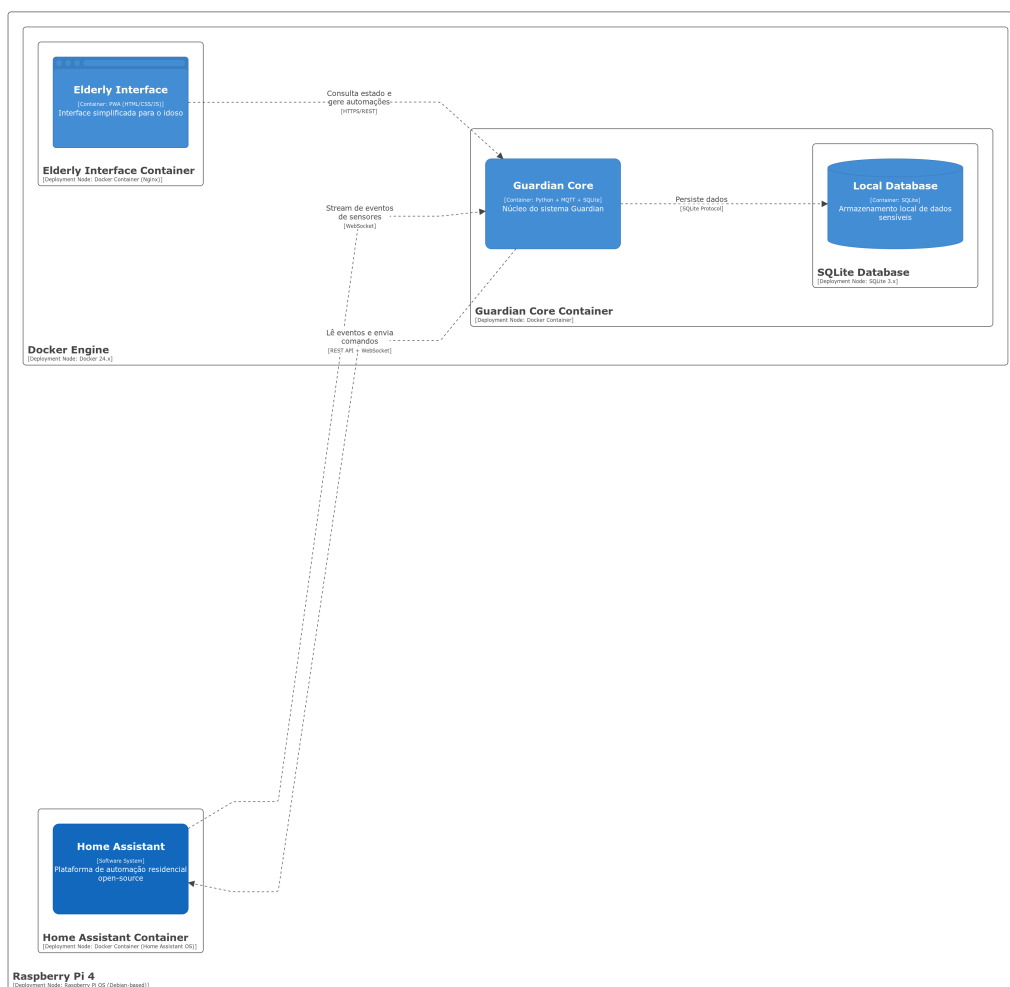


Figura 4: Visão do Componente do Sistema (C3)



Deployment View: Guardian System - Residência do Idoso
 Vista de implantação do sistema Guardian numa residência típica, mostrando a execução em Raspberry Pi com containers Docker
 quinta-feira, 15 de janeiro de 2026 às 18:29 Hora padrão da Europa Ocidental

Figura 5: Vista de Deployment do Sistema Guardian